

[Domain adaptation](#) · [visual guide](#)

CSRDA: cyclic synthetic-to-real domain adaptation

A visual walkthrough of the method and its reproduction — from the outer loop down through each mechanism, then the practical pipeline that produces the numbers.

► Table 1 · $\hat{S}2C$ task · source = HKU-IS (synthetic) →
target = COD10K (real) · eval = COD10K-test

CSRDA is a mean-teacher domain-adaptation method wrapped in a cyclic outer loop. It bridges the gap between labeled synthetic images and unlabeled real ones for camouflaged object detection. The sections below build the picture top-down — the loop, then the training step inside it, then the two custom pieces (the consistency loss and the pseudo-label selector) — and finish with the data layout and the four-phase reproduction run.

Key settings for the $\hat{S}2C$ task (hard-coded as a task override, not passed on the CLI)

Symbol	Meaning	Value
λ	teacher EMA smoothing	0.996
μ	CLS agreement multiplier	0.8
τ	CLS confidence threshold	0.4
a, b, c	edge / region / saliency-offset weights	0.9, 0.3, 0.5
—	optimizer / lr / batch / trainsize	Adam · 1e-4 · 16 · 352
—	epochs · rounds · decay	40 · 2 · ×0.1@30

The cyclic CSRDA loop

One invocation runs `--iteration` rounds (default 2). Each round is a full mean-teacher training run that starts from **fresh ImageNet weights** — round 2 does not fine-tune round 1. What carries forward is only the *data*: between rounds, CLS harvests confident predictions on the real target domain and folds them into the source set, so the supervised loss itself begins pulling the model toward the target domain instead of leaving that job entirely to the consistency term.

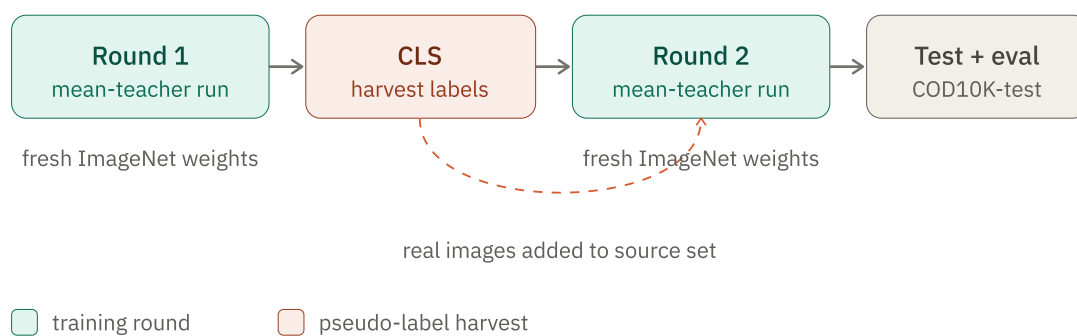


Figure 1. The outer loop. Weights reset each round; only pseudo-labeled data carries forward.

02 · Inside one round

The mean-teacher training step

On every batch, two things happen in parallel. A **supervised** pass runs labeled source images through the student, producing L_{sup} (a BCE on both SINet outputs). A **consistency** pass takes the same target image in two augmentations: the teacher sees the weak view (resize + normalize) and the student sees the strong view (adds RandomAutocontrast and GaussianBlur), sharing one random horizontal flip. The teacher's map supervises the student's through $L_{\text{con}} = \text{ESLoss}$.

The total loss $L = L_{\text{sup}} + L_{\text{con}}$ updates the student with Adam. The teacher never sees a gradient — it only ever updates as a slow EMA of the student, $\phi \leftarrow \lambda\phi + (1-\lambda)\theta$ with $\lambda = 0.996$. One subtlety worth knowing: the EMA covers parameters only, not BatchNorm buffers, and the teacher runs in train() mode on target images — so it quietly accumulates target-domain batch-norm statistics.

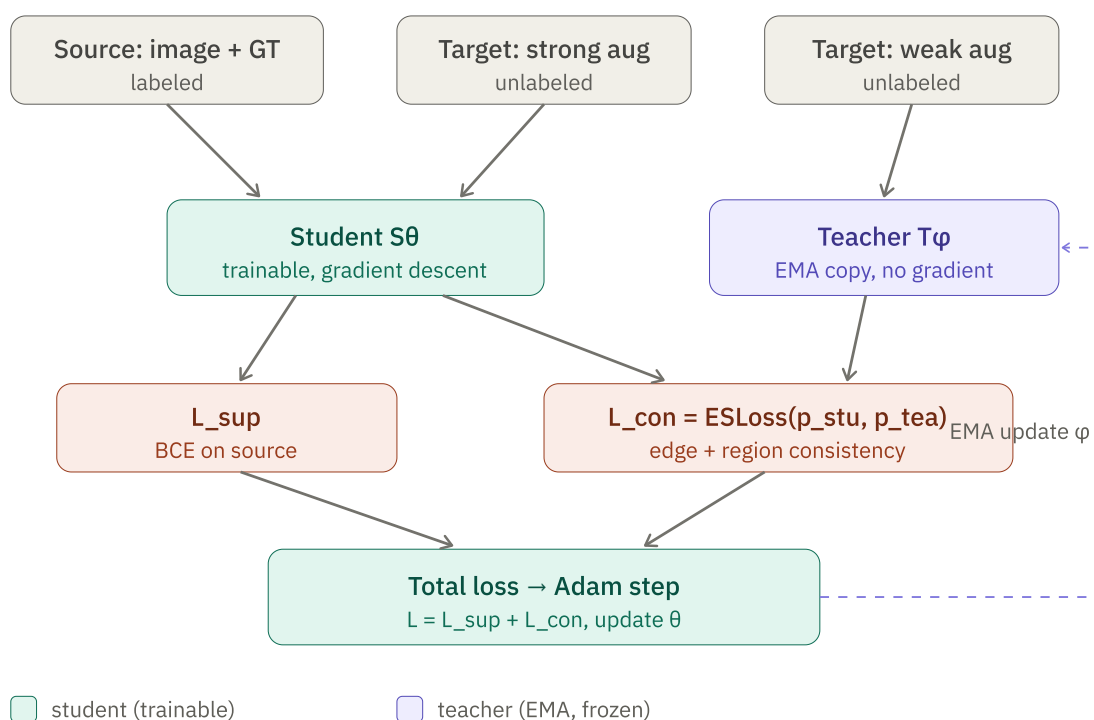


Figure 2. One step. Student learns by gradient descent; teacher trails it as an EMA.

03 · The consistency loss

Anatomy of ESLoss

$\text{ESLoss} = a \cdot \text{edge_loss} + b \cdot \text{region_loss}$, with $a = 0.9$ and $b = 0.3$. The **edge** term is the L1 distance between the Sobel gradient magnitudes of the two maps; the **region** term is a BCE between them weighted by $p_{\text{tea}} + c$ ($c = 0.5$), which upweights pixels the teacher already believes are object. Because a is roughly three times b , boundary agreement dominates the signal — and boundaries are exactly where synthetic→real transfer degrades most, which is what makes the loss camouflage-aware.

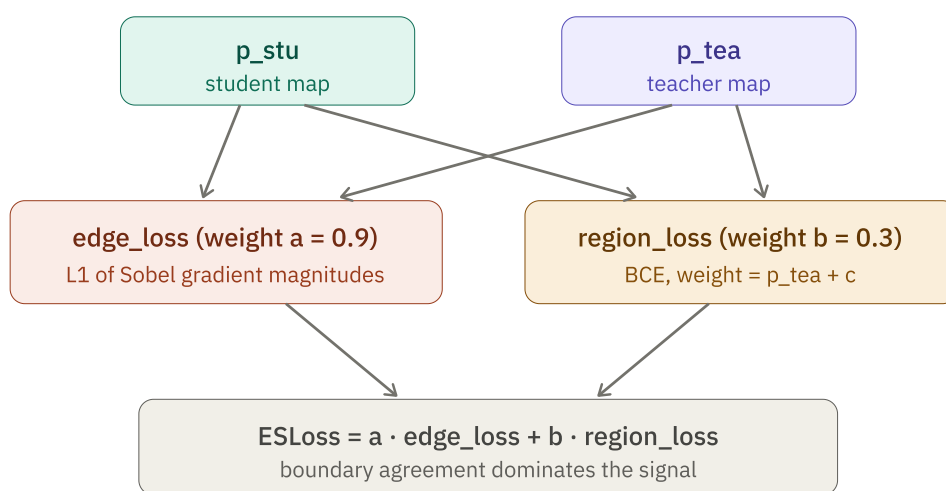


Figure 3. Two comparisons of the same pair of maps; the edge term is weighted heavier. Note: CLS scoring uses an *unweighted*-BCE variant so its ranking isn't biased by predicted object area.

CLS — the two-gate filter

Between rounds, CLS decides which real target images are trustworthy enough to become supervised data. It loads the trained student (`Stu_40.pth`) and the selected teacher (`Tea_epoch_best.pth`). **Pass 1** measures, for every target image, how much the two disagree — $d(x)$, an unweighted ES Loss — and its dataset mean. **Pass 2** admits an image only if it clears both gates: the *agreement gate* ($d(x) < \mu \cdot \text{mean}$, $\mu = 0.8$), a relative self-calibrating threshold, and the *saturation gate* (teacher confidence $\text{max} \geq \tau$, $\tau = 0.4$). Kept maps are thresholded — everything below τ zeroed — then min-max renormalized and saved as soft-mask pseudo-GTs into the next source set.

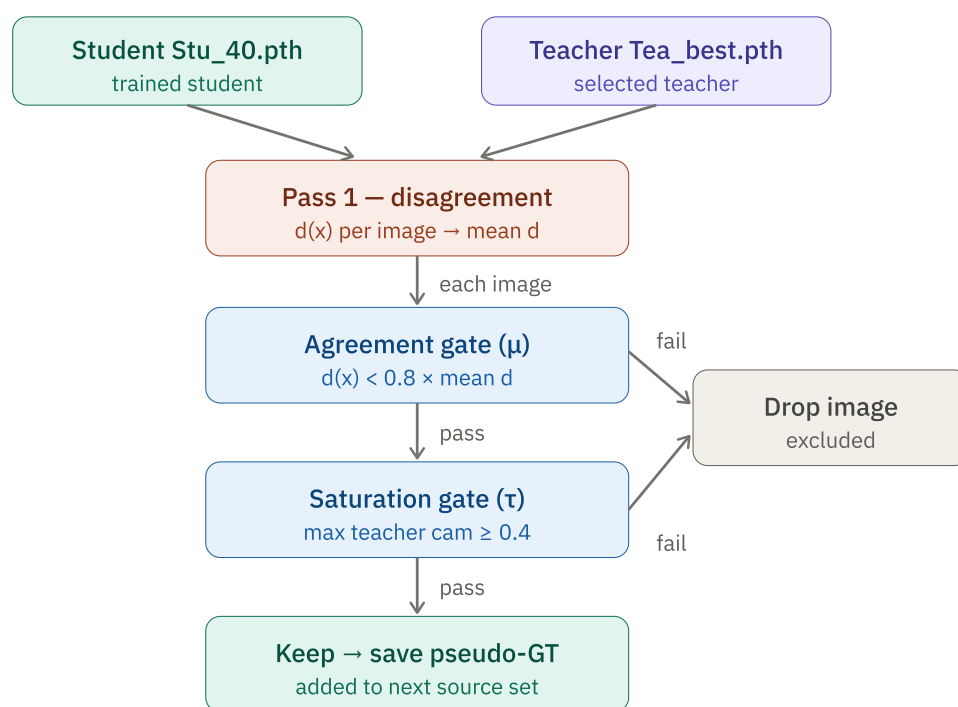


Figure 4. A pseudo-label must clear both gates. Failing either drops the image.

Dataset layout and roles

The final `Dataset/` layout carries four roles: labeled synthetic source (4447 pairs), unlabeled real target (4040 images), a real validation set for model selection (250 pairs), and the real test set (2026 image/GT pairs). Two archives — `Target.rar` and `Test.rar` — ship unextracted and **nest their own top-level folder**, so you extract into `Dataset/`, not into the subfolder, or the loaders fail. The `HKU-IS_iteration2/` directory is created by CLS at runtime; never build it by hand. Steps per epoch settle at $\min(\lceil 4447/16 \rceil, \lceil 4040/16 \rceil) = 253$.

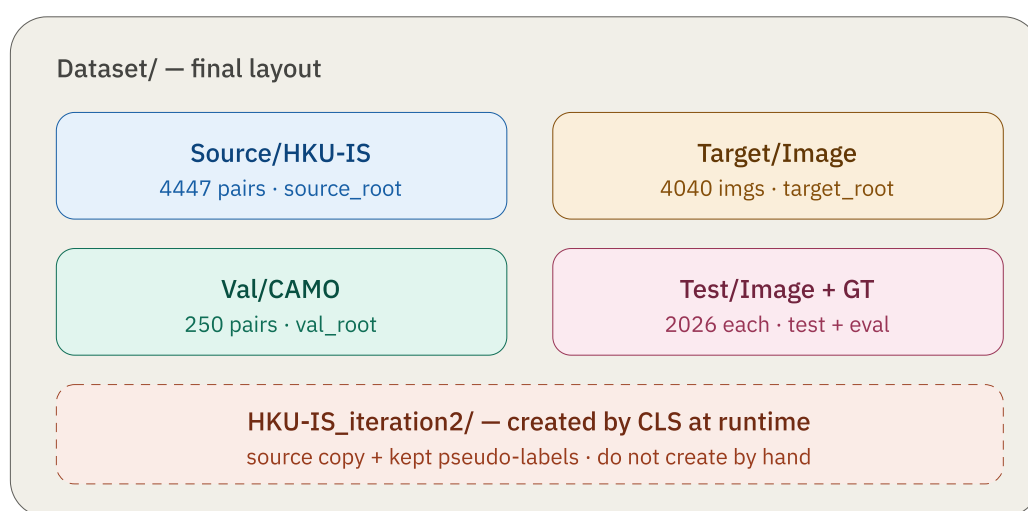


Figure 5. Folder roles. The dashed box is generated at runtime, not authored.

The reproduction pipeline

Four phases. **Setup**: add the missing dependencies (`scipy` , `opencv-python` , `scikit-learn` , `prettytable` , `tqdm`) and sync, download the ResNet-50 backbone, and extract the datasets. **Train**: a single `MyTrain.py` invocation runs both CSRDA rounds and the CLS step between them — you never launch round 2 by hand. **Test**: run `MyTest.py` with the final round's teacher (`Tea_epoch_best.pth`) to produce 2026 masks. **Eval**: run `MyEval.py` from inside `Eval/` and compare `_eval.txt` to Table 1. Every path argument must end in a trailing slash.

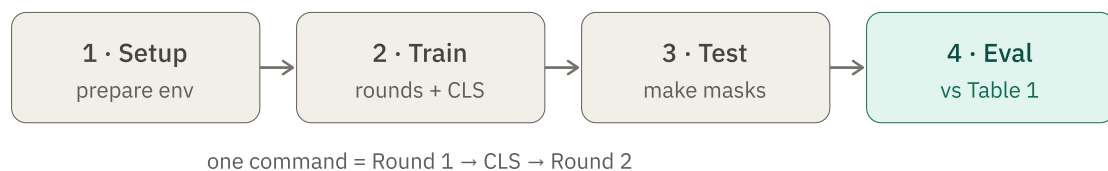


Figure 6. Setup → train → test → eval. Phase 2 runs the entire loop from Figure 1.

Reference

Three gotchas worth remembering

These live in the upstream code and match the settings that produced the published numbers — so for reproduction, change nothing. They're listed so nothing surprises you.

schedule

The LR schedule compounds. From epoch 30 the learning rate is multiplied by 0.1 *every* epoch, not once — so it falls $1e-5 \rightarrow 1e-6 \rightarrow \dots$ and training is effectively frozen after about epoch 31. In practice, `Tea_epoch_best.pth` is selected from epochs 21–31.

off-by-one

39 epochs, not 40. The loop runs `range(1, 40)`, and the checkpoint named `Tea_40.pth` is saved at the final step only because saving uses `epoch + 1`. CLS depends on that exact filename, so don't change `--epoch`.

the dangerous one

The CLS working directory is shared by every network — and CLS deletes it. Both SINet and SINet-v2 write to `HKU-IS_iteration2/`, because the S2C task block resets `source_root` after argparse and discards your `--source_root`.

Sequential runs are safe — CLS wipes and rebuilds from the pristine source — but the earlier network's pseudo-label set is lost unless you rename the directory between runs. **A concurrent run is a destructive race:** one process's delete pulls files out from under the other mid-epoch, giving a `FileNotFoundError` or, worse, silently mixed pseudo-labels. Run the two networks one after another, and rename `HKU-IS_iteration2` after each.

CSRDA visual guide · S2C task (HKU-IS → COD10K) · diagrams render in light and dark mode and are print-safe (File → Print → Save as PDF).